

MINI PROYECTO 4

Sistema de Juego de Preguntas por Niveles con Java Swing y MVC

Autor: Cristian Camilo Cortes Ortiz
Programa: Tecnología en Desarrollo de Software
Universidad del Valle
Asignatura: Programación Orientada a Eventos
Docente: Daniel Esteban Gaviria Clavijo
Fecha: 5 Diciembre de 2025
cristian.cortes.ortiz@correounivalle.edu.co

Resumen

Este informe presenta el desarrollo del MiniProyecto 4, un videojuego educativo basado en preguntas de opción múltiple, desarrollado en Java utilizando Swing bajo el patrón de diseño Modelo–Vista–Controlador (MVC).

El sistema implementa tres niveles de dificultad, un mecanismo de temporización por pregunta, administración de puntajes con ScoreManager, reordenamiento dinámico de respuestas mediante algoritmos de mezcla, validación de entrada del usuario, y una interfaz gráfica.

El objetivo del proyecto es aplicar conceptos de programación orientada a eventos, diseño modular, reutilización de código y buenas prácticas de desarrollo de software en el contexto de aplicaciones interactivas.

Palabras clave —Java, MVC, videojuego, Preguntas de opción múltiple

I. Introducción

Los sistemas educativos interactivos han tomado relevancia por su utilidad como herramientas de refuerzo cognitivo. El objetivo de este proyecto fue desarrollar un juego de preguntas y respuestas, implementando conceptos fundamentales de POO, programación orientada a eventos, MVC y GUI en Java.

Este proyecto sirve como una aplicación práctica para comprender:

- Gestión de interfaces gráficas (Swing, AWT).
- Programación dirigida por eventos.
- Modelamiento mediante clases y objetos.
- Controladores para la manipulación del flujo del juego.
- Timers, listeners y validación de estados.
- Persistencia simple de datos para puntuaciones.

II. Desarrollo del Sistema

A. Arquitectura General

El sistema se desarrolló siguiendo el patrón Modelo – Vista – Controlador (MVC):

1. Modelo (Model)

Incluye las clases responsables de la lógica del juego:

- Question: estructura de preguntas, opciones y respuesta correcta.
- ScoreManager: almacenamiento y lectura de puntajes.
- User: registro y validación de usuario.

2. Vista (View)

Interfaz desarrollada con Java Swing, que incluye:

- StartView
- RegisterView
- CategoriesView
- GameView
- ResultView
- WinView

Las vistas comparten un estilo visual consistente derivado de paneles con gradientes, tipografías futuristas y botones personalizados.

3. Controlador (Controller)

Administra la lógica del juego:

- Carga de preguntas.
- Control de niveles.
- Validación de respuestas.
- Aleatorización de opciones.
- Manejo del temporizador por pregunta (Swing Timer).
- Navegación entre vistas.

B. Funcionalidades Implementadas

1. Progreso por niveles

El juego incluye 3 niveles:

Nivel	Preguntas	Dificultad	Puntaje mínimo
1	10	Fácil	6
2	10	Media	6
3	10	Difícil	6

El usuario avanza automáticamente si supera el puntaje mínimo.

Si completa el Nivel 3 → gana el juego.

2. Sistema de temporización

Cada pregunta tiene 15 segundos administrados mediante `javax.swing.Timer`.
Si el tiempo se agota:

- Cuenta como respuesta incorrecta
- Avanza a la siguiente pregunta automáticamente

Esto añade presión y realismo al sistema de evaluación.

3. Reordenamiento dinámico de respuestas

- Para evitar patrones, las opciones se mezclan automáticamente:
- Se clona la lista original.
- Se implementa un shuffle controlado.
- Se recalcula dinámicamente el índice de la respuesta correcta.

Este enfoque evita que la respuesta correcta permanezca siempre en la primera posición.

4. Persistencia de puntajes con ScoreManager

Se implementa almacenamiento simple mediante archivos:

- Nombre del jugador
- Puntaje obtenido
- Nivel alcanzado

Esto permite mostrar resultados previos y mantener un historial básico.

5. Interfaz gráfica

El diseño del proyecto incluye:

- Gradientes con `Graphics2D`
- Botones personalizados
- Tipografía Consolas
- Marcos centrados automáticamente
- Colores neón consistentes

La GUI fue un requisito fundamental para mantener coherencia estética.

6. Botón para terminar el juego

El usuario puede abandonar la partida en cualquier momento mediante un botón dedicado en `GameView`, manejando adecuadamente el estado del controlador.

C. Programación Orientada a Eventos

El núcleo del juego depende de la gestión de eventos:

- ActionListener para botones.
- MouseAdapter para animación visual.
- Timer para manejo del tiempo.
- Validación de selección mediante estados del modelo.

Esto permitió coordinar:

- Avance entre preguntas
- Confirmación de respuesta
- Retroalimentación visual
- Control de niveles

III. Resultados

El sistema final presenta:

- Flujo completo de registro → categoría → juego → resultados → avance de nivel.
- Comportamiento estable incluso si el usuario hace clic repetidamente en los botones.
- Preguntas correctamente temporizadas.
- Aleatorización robusta de opciones.
- Estética visual consistente.
- Tres niveles con incremento de dificultad y pregunta ganadora.

El usuario obtiene una experiencia fluida, visualmente atractiva y funcional.

IV. Conclusiones

El desarrollo del MiniProyecto permitió comprender:

- La importancia del patrón MVC, que facilita la extensibilidad del software.
- Programación orientada a eventos como pilar central en interfaces gráficas.
- Manejo adecuado de Timers para lógica dependiente del tiempo.
- Implementación de técnicas de aleatorización para mejorar la dinámica de juego.
- Diseño e integración de interfaces visuales avanzadas con Swing.
- La necesidad de validar estados para evitar errores lógicos (clics múltiples, no seleccionar opción, etc.).

Además, la separación modular del sistema permite escalar fácilmente el proyecto para agregar más niveles, categorías o mecánicas.

V. Referencias

1. Oracle, "The Java™ Tutorials – Creating a GUI With Swing," docs.oracle.com. Disponible en: <https://docs.oracle.com/javase/tutorial/uiswing/>
2. Oracle, "Class Timer (Java Platform SE)," docs.oracle.com. Disponible en:
3. <https://docs.oracle.com/javase/7/docs/api/javax/swing/Timer.html>
4. Gamma, E., Helm, R., Johnson, R., Vlissides, J., Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994.